MSML606 – Project 2 Proposal

SkillPath: Career Skill Dependency Mapper

1. Team Composition:

Monark Dixit – Individual Submission | UID: 122259645

2. Overview:

The goal of **SkillPath** is to solve a specific problem I encounter constantly: job postings list dozens of requirements, but they fail to explain the logical order in which you should learn them. When looking at a role like "Machine Learning Engineer," seeing Python and System Design side-by-side is confusing because one is a foundation for the other.

I am building this application based on the observation that professional skills form a Directed Acyclic Graph (DAG). You cannot meaningfully learn TensorFlow without NumPy, just as you cannot perform System Design without understanding databases. My project maps these hidden dependencies to take the guesswork out of career transitions. By modeling skills this way, I can use Topological Sorting to give users a valid learning sequence and Shortest Path algorithms to show them the most efficient route from their current skill set to their dream job.

3. Dataset:

I plan to use the LinkedIn Job Postings 2023 dataset, available on Kaggle:
https://www.kaggle.com/datasets/arshkon/linkedin-job-postings

It contains over 33,000 real job postings with structured skill tags and role titles. I will work with two files:

1. job skills.csv, which maps job IDs to required skills, and
2. job summary.csv, which maps job IDs to role titles.

Joining them gives me a clean mapping of “role” -> “required skill set”, which I will use for both frequency analysis and graph construction.

For the skill dependency edges, i.e, which skills are prerequisites for which others, I will define these myself as a curated list of roughly 40–60 directed pairs (e.g, Python -> Pandas, Statistics -> Machine Learning). No public dataset encodes these relationships explicitly, so building this edge list is one of the original contributions I am making in this project.

3. Application:

I will build SkillPath as a Streamlit app. The user provides two inputs: a checklist of skills they already have, and a dropdown to pick a target job role. From there, the app will:

1. Build the skill dependency DAG from my curated edge list
2. Mark the user's existing skills as already-acquired nodes
3. Run BFS from the user's current skills toward those required by the target role, finding the minimum number of prerequisite steps
4. Apply topological sort to the resulting subgraph to determine the correct learning order
5. Return a numbered roadmap: the fewest skills to acquire, in the right sequence, to qualify for the role

I also want to show a visual of the dependency graph for the target role, a tally of skills already met vs. still needed, and a frequency ranking of the most commonly required skills across all postings for that role.

4. Algorithm:

I will implement three graph algorithms from scratch:

1. **Topological Sort** (Kahn's algorithm). I will process the DAG using in-degree counting and a queue. Nodes with no unresolved prerequisites get enqueued first; as each is processed, I decrement the in-degrees of its neighbors, and any that hit zero join the queue. The order they exit is a valid learning sequence that respects all prerequisite relationships.
2. **BFS Shortest Path on the DAG**. Starting from the user's current skill set, I propagate BFS forward through the dependency graph. Each layer represents one prerequisite step. This gives me the minimum number of skills the user needs to acquire to reach everything required by their target role.
3. **DFS Cycle Detection**. I will run this at graph construction time to confirm the graph is actually acyclic. A cycle, something like "Skill A requires Skill B requires Skill A", is logically impossible and would break topological sort. If one is found, I will flag and remove the offending edge before any computation runs.

4. Scope and AI Usage:

I will write all core algorithm code manually, graph construction, topological sort, BFS, and cycle detection. The one place I plan to use AI assistance is the Streamlit UI layer, which is permitted under course policy. I will document all such usage clearly in an AI USAGE DISCLOSURE.md file included in my repository.